

T07 — Librerie Standard e Manipolazione delle Stringhe

Java Class Library, Immutabilità della Classe String, Metodi Fondamentali e SimpleDate v2

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo la gestione delle librerie standard e la classe String ([T07 - Libraries and String Class.pdf](#)), insieme all'evoluzione del codice nella cartella [Lezione07](#), dove il docente introduce **incapsulamento (private)**, **costruttori sovraccaricati con delega this(...)**, **copy constructor**, formattazione internazionalizzata e il primo test con **asserzioni (assert e flag -ea)**.

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T07)

A. Librerie Standard e Meccanismo di Import (Slide 1–9)

- **Librerie Standard della JDK:**

- `java.lang`: classi fondamentali del runtime (oggetti base, tipi primitivi wrapper, stringhe, thread, sistema). **È l'unico package importato automaticamente e implicitamente in ogni file sorgente Java.**
- `java.util`: strutture dati (collezioni), utilità temporali, parser e scanner.
- `java.io`: gestione di stream di input/output e filesystem.
- `java.math`: calcolo a precisione arbitraria (`BigInteger`, `BigDecimal`).

- **Sintassi di Importazione:**

- Selettiva: `import java.util.Scanner;` (consigliata per chiarezza e pulizia).
- Wildcard su package: `import java.util.*;` (rende visibili tutte le classi pubbliche del package).
- *Regola del Classpath*: le classi definite dall'utente che risiedono nello stesso package e nella stessa cartella del Classpath non necessitano di alcuna direttiva import.

- **Classi di Sistema Notevoli:**

- `java.lang.System`:
 - * Tre stream predefiniti: `System.in` (standard input, byte stream da tastiera), `System.out` (standard output a video), `System.err` (standard error).
 - * `System.currentTimeMillis()`: restituisce un long con il timestamp **Unix Epoch** (millisecondi trascorsi dalle ore 00:00:00 UTC del 1° Gennaio 1970).
- `java.util.Scanner`:
 - * Parser per estrarre tipi primitivi e stringhe da flussi di testo:
 - `sc.nextLine()`: legge un'intera riga fino al terminatore `\n` e restituisce una `String`.

- `sc.nextInt()`, `sc.nextFloat()`, `sc.nextDouble()`: leggono il token successivo effettuando il parsing del tipo. Se l'utente inserisce caratteri incompatibili, lancia un'eccezione (`InputMismatchException`).
 - * Buona norma: invocare `sc.close()` per rilasciare il descrittore del file o dello stream ed evitare *resource leak*.
-

B. La Classe String e le sue Operazioni (Slide 10–26)

- **Immutabilità:**

- Le istanze di `String` sono immutabili. Nessun metodo della classe altera i caratteri dell'oggetto esistente: qualunque operazione (es. `replace`, `substring`) restituisce **una nuova istanza di String** nello Heap.
- Tentare `str.charAt(1) = 'c'`; produce un **Compile-Time Error**.

- **Operazioni Fondamentali:**

- `str.length()`: numero di caratteri (nota: è un metodo con parentesi tonde, a differenza di `arr.length` per gli array).
- `str.charAt(int index)`: carattere in posizione `index` (0-indexed). Indici negativi o $\geq \text{length}()$ lanciano `StringIndexOutOfBoundsException`.
- `str.indexOf(char c)`: prima occorrenza del carattere (restituisce -1 se non presente).
- `str.substring(int start, int end)`: estrae la sottostringa nell'intervallo semi-aperto $[start, end)$, ovvero il carattere all'indice `end` è **escluso**. Se `end` è omissso (`str.substring(start)`), estrae fino al termine della stringa.
- `str.replace(CharSequence target, CharSequence replacement)`: sostituisce tutte le occorrenze da sinistra a destra.
- `String.format("Format: %s, %d", str, num)`: formattazione con sintassi analoga alla `printf` del C.

- **Confronto di Stringhe: `==` vs `.equals()`:**

- `s == t`: confronta l'**identità dei puntatori** (indirizzi nello Heap). È `true` solo se `s` e `t` puntano allo stesso identico oggetto.
- `s.equals(t)`: confronta l'**equivalenza semantica** dei contenuti carattere per carattere.
- **Lo String Constant Pool e l'ottimizzazione del compilatore:**

```
1 String s = "hello";
2 String t = "hello";
3 System.out.println(s == t); // Stampa TRUE!
```

Perché? Le costanti letterali vengono inserite nel *String Constant Pool*. Se due variabili dichiarano lo stesso letterale, la JVM riusa lo stesso identico oggetto in memoria per risparmiare RAM (*interning*). Tuttavia, se scriviamo `String r = new String("hello");`, costringiamo la JVM a creare un nuovo oggetto nello Heap: `s == r` sarà `false`! **Regola d'oro: usare SEMPRE `.equals()`.**

C. Stringhe Mutabili con `StringBuilder` (Slide 27–29)

- **Il problema delle prestazioni con `String`:**

- L'operatore `+` tra stringhe invoca internamente `String.valueOf()`. Se usato in un ciclo di N iterazioni, alloca $O(N)$ oggetti temporanei che intasano la memoria del Garbage Collector.

- **La soluzione: `StringBuilder`:**

- Mantiene un buffer interno ridimensionabile mutabile.
- Metodi principali: `.append(...)`, `.delete(start, end)`, `.insert(index, str)`, `.reverse()`.
- Supporta la **Fluent Notation (Method Chaining)** perché restituisce `this`.

0.0.2 2. Codice: Evoluzione del Codice: SimpleDate (Lezione 07)

Nel passaggio da Lezioni05-06 a Lezione07, il docente trasforma radicalmente la classe `Date.java` applicando i principi di Information Hiding e aggiunge per la prima volta una classe di test dedicata: `TestDate.java`.

Analisi: Il Diff Concettuale di `Date.java`:

```
1 public class Date {
2     - int day;
3     - int month;
4     - int year;
5     - static final String FORMAT = "dd/mm/yyyy";
6 + private int day;
7 + private int month;
8 + private int year;
9 + static final String FORMAT_IT = "dd/mm/yyyy";
10 + static final String FORMAT_US = "mm/dd/yyyy";
11 + private byte lang; // 0: IT, 1: US
12
13 // Costruttore principale
14 public Date(int day, int month, int year) {
15     this.day = day;
16     this.month = month;
17     this.year = year;
18     verify();
19 +     lang = 0;
20 }
21
22 + // Costruttore sovraccaricato: delega tramite this(...)
23 + public Date(int day, int month) {
24 +     this(day, month, 2025); // default year
25 + }
26
27 + // Copy Constructor: clona lo stato da un'altra istanza
28 + public Date(Date other) {
29 +     this.day = other.day;
30 +     this.month = other.month;
31 +     this.year = other.year;
32 +     verify();
33 +     lang = other.lang;
34 + }
35
36 + // Getters
37 + public int getDay() { return day; }
38 + public int getMonth() { return month; } // Attenzione: Attenzione al refuso del prof!
39 + public int getYear() { return year; } // Attenzione: Ritorna 'year' anziché 'year'!
40 + public byte getLang() { return lang; }
41
42 + // Setters con validazione dell'invariante
43 + public void setDay(int day) {
44 +     this.day = day;
45 +     verify();
46 + }
47 + public void setMonth(int month) {
48 +     this.month = month;
49 +     verify();
50 + }
51 + public void setYear(int year) {
```

```
52 +     this.year = year;
53 +     verify();
54 + }
55 + public void setLang(byte lang) {
56 +     if (lang == 1) this.lang = 1;
57 +     else this.lang = 0;
58 + }
59 +
60 + public String printFormat() {
61 +     if (lang == 0) return FORMAT_IT;
62 +     else return FORMAT_US;
63 + }
64
65 + public String toString() {
66 -     return print();
67 +     if (lang == 0) return day + "/" + month + "/" + year;
68 +     else return month + "/" + day + "/" + year;
69 }
```

Nota: Le Novità e le Finezze Implementative da Notare:

1. Incapsulamento e Protezione dello Stato (`private`):

- I campi non sono più accessibili direttamente dall'esterno (`d.day = -7`; genera ora un **Compile-Time Error** in `MainDate`).

2. Delega tra Costruttori con `this(...)`:

- Il costruttore `Date(int day, int month)` invoca `this(day, month, 2025)`.
- **Regola tassativa per l'esame:** la chiamata `this(...)` verso un altro costruttore deve essere **la prima istruzione assoluta** del costruttore, pena errore di compilazione («*Call to 'this()' must be first statement in constructor body*»).

3. Copy Constructor:

- `public Date(Date other)` permette di creare una nuova istanza duplicando i valori di un oggetto esistente, evitando l'aliasing accidentale dei puntatori.

4. Validazione Continua dell'Invariante nei Setter:

- Ogni volta che un setter muta lo stato (`setDay`, `setMonth`, `setYear`), invoca subito `verify()`. Lo stato dell'oggetto viene così monitorato costantemente.

5. Il Refuso di Copia-Incolla del Docente nei Getter:

- Nei getter di `Date.java` (righe 38–39 del codice ufficiale), il prof ha inavvertitamente scritto:

```
1 public int getMonth() { return day; }
2 public int getYear() { return day; }
```

Sia `getMonth()` che `getYear()` restituiscono il campo `day` anziché `month` e `year`! Nel tuo codice assicurati di restituire i campi corretti (`return month`; e `return year`).

6. Testing con Asserzioni e Flag `-ea` (`TestDate.java`):

```
1 public class TestDate {
2     private static Date date = new Date(1, 1, 1970);
3
4     private static void printFormatTest() {
5         date.setLang((byte) 1);
6         assert date.printFormat().equals(Date.FORMAT_US);
7     }
8
9     public static void main(String[] args) {
10         printFormatTest();
11     }
12 }
```

- **Concetto Chiave d'Esame:** La keyword `assert` verifica che una condizione booleana sia vera; se è falsa, solleva un `AssertionError`.
- **Attenzione:** Nella JVM le asserzioni sono **disabilitate di default**. Se lanci semplicemente `java TestDate`, l'istruzione `assert` viene ignorata. Per attivarle, è obbligatorio passare il flag `-ea` (*Enable Assertions*):

```
1 java -ea TestDate
```

- Nel tuo script `esercizi.sh`, infatti, il comando `./esercizi.sh test` usa proprio `-ea`!

0.0.3 3. Ciclo: Corrispondenza con la Struttura del tuo Workspace

Nel tuo repository `esercizi-wally-25-26`, hai modularizzato perfettamente questi argomenti: - **es02:** `MainScanner.java` copre l'uso di `java.util.Scanner` introdotto in T07. - **es04:** `MainString.java` testa tutti i metodi di `String` (`length`, `charAt`, `indexOf`, `substring`, `replace`, `constant pool ==` vs `.equals()`) e le mutazioni con `StringBuilder` (`append`, `delete`, `insert`, `reverse`). - **es05:** `Date.java` implementa l'overloading dei costruttori, il constructor chaining con `this(...)` e il copy constructor.

Nota di Studio

Con la **Lezione 07** abbiamo completato le librerie standard, le stringhe, i costruttori multipli e le asserzioni.

Il prossimo blocco è la **Lezione 08 / Slide T08: Class Constructors and Encapsulation**: - Studio approfondito della teoria su costruttori (default constructor, costruttore falso, copy constructor, overloading). - Analisi dettagliata dell'Information Hiding e dei **modificatori di accesso** (`public`, `protected`, `package-private` di default, `private`). - Evoluzione del codice in Lezione08: sostituzione del `byte lang` con un tipo enumerato ad hoc: `Language.java` (`enum Language { IT, US }`)! - Array costanti per i nomi dei mesi (`MONTHS_IT`, `MONTHS_US`) e nuovo metodo `getMonthAsString()`.

Dammi conferma quando vuoi procedere alla Lezione 08 / T08!